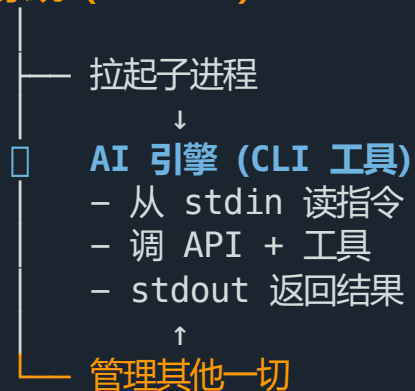


AI 引擎是商品，Harness 才是产品

你的系统 (Harness)



Engine = commodity.

Harness = your IP.

- AI 引擎 = 无状态工人：没有记忆、不知道昨天、不会自愈
- 记忆 / 自愈 / 生命周期 / 从错误中学习 —— 全部来自 Harness
- 模型可互换；围绕它建的判断系统才是你的 IP

像理解汽车一样理解 Agent

汽车组件	Agent 组件	做什么
发动机	AI CLI (Claude、Kiro 等)	原始动力
传感器	Context 文件、工具、MCP	感知世界
控制系统	Pipeline、路由逻辑	即时决策
导航系统	目标分解、状态管理	规划多步旅程
整辆车	Agent Harness	所有部件协同

引擎可以换。特斯拉换电机版本 ≈ AgentOS 换 Claude 为 Gemini —— 围绕引擎的系统才是产品。

诚实说：设计理念可迁移，但具体实现（hooks 格式、settings 结构、CLI 行为）是 Claude Code 特有的。换引擎 = 重接 harness 接口。

从 L1 到 L5：你在哪，你要去哪

级别	名称	Agent 做什么	你做什么	代表产品
L1	辅助	单次调用，展示结果	决定问什么、何时问	ChatGPT 网页版
L2	副驾驶	观察 + 建议补全	接受或拒绝	Copilot、Cursor Tab
L3	智能体	多步推理 + 工具 + 检查点	设目标、审查、批准	Claude Code、Kiro
L4	自主	+ 无人值守 + 记忆 + 自愈 + 并发	设意图、偶尔检查	SwarmAI、Devin
L5	自进化	+ 跨天规划 + 从错误学习 + 扩展自身	设方向	2026 年无完整 L5

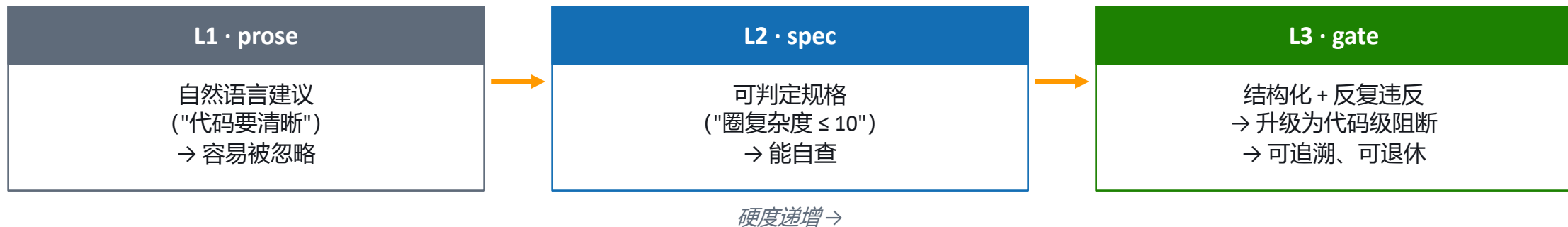
每一级是累积的——L4 包含 L1-L3 全部能力

能力	L1	L2	L3	L4	L5
执行单次工具调用	✓	✓	✓	✓	✓
自动注入相关上下文		✓	✓	✓	✓
多步推理 + 工具使用			✓	✓	✓
跨 session 记忆				✓	✓
无人值守运行				✓	✓
崩溃后自动恢复				✓	✓
同时处理多任务				✓	✓
随时间改进自身行为					✓
把大目标拆成多天计划					✓
L3→L4 跃迁需要：记忆 + 无人值守 + 自愈 · 本课覆盖：Knowledge（记忆）+ Engine（无人值守流程）+ Loop（自愈）					

基座提供六种能力，模块组合使用它们

能力	做什么	被谁用（举例）
Rules	文本约束：从 prose 到可判定 spec	Knowledge → Governance 产出物
Hooks	事件触发的自动执行	Knowledge → capture（SessionEnd） Engine → gate（PreCommit）
Skills	按需加载的工作流包	Engine → 某 stage 执行逻辑 Knowledge → distill / evolve
MCP	外部系统能力接入	Engine → 需要的领域工具
Context	上下文组装与预算管理	Knowledge → SessionStart 注入
Sessions	会话生命周期管理	Engine → State 持久化、中断恢复

Rules = 文本约束的光谱 (L1 prose → L2 可判定 spec)

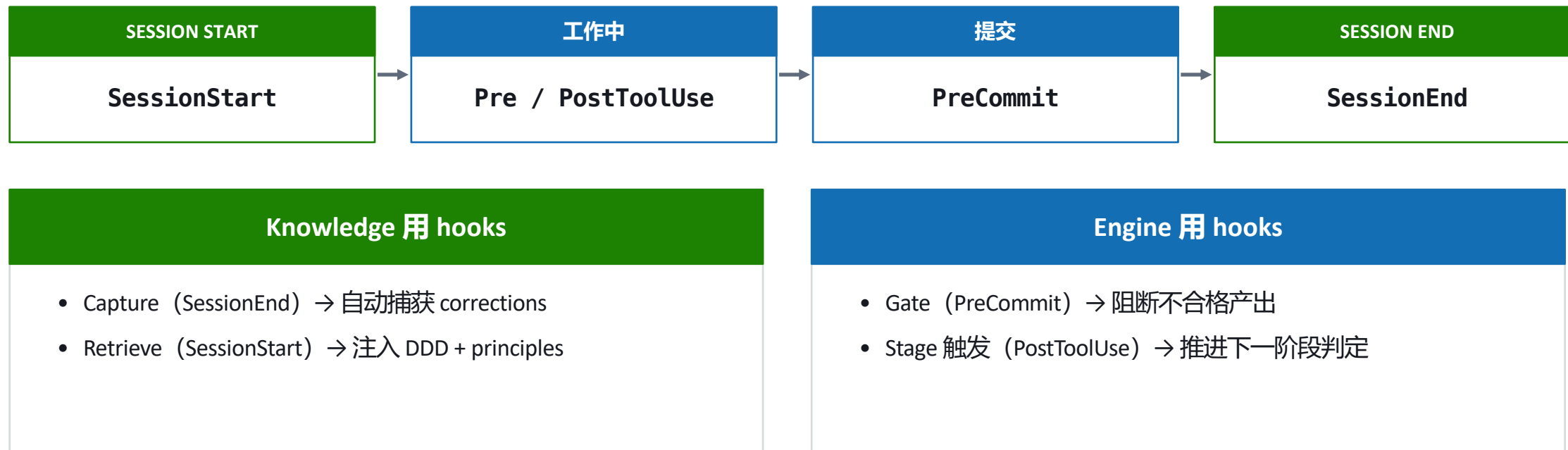


在 AgentOS 中的位置

- Rules 是 Knowledge / Governance 的产出物 —— 不是你直接手写的东西
- 从 corrections 中蒸馏而来: agent 犯错 → 你纠正 → 积累 → 发现 pattern → 蒸馏成 rule
- 每条必须 link to principle、带证据、有过期条件 (gate 覆盖后即退休)

关键心智转变: 从「我来写规则管 agent」 → 「系统蒸馏出规则、我来审批」。

Hooks = 事件驱动的自动化（Agent 的神经反射）



Hook 把「我记得要做这件事」变成「系统自动做这件事」——这是 L1（人记得）升级为 L3（自动执行）的通道。

Skills = 按需加载的工作流包 (Agent 的专业技能)

my-skill/

SKILL.md	触发条件 + 执行流程 + 产出规范
scripts/	可选脚本
templates/	可选模板

渐进式加载 (lazy load) :

- 触发时才载入, 用完释放
- context 预算 100K — 装不下全部
- 不是常驻, 是按需唤醒

Engine 的 Stage

每个执行阶段可以是一个 skill (BUILD → coding skill)

Knowledge 的进化

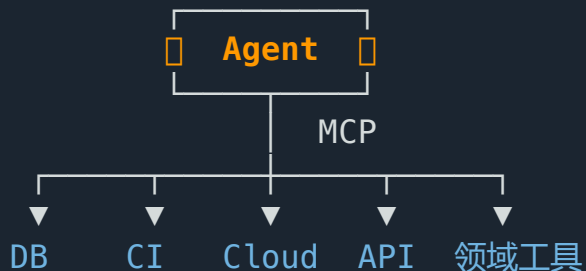
distill / evolve 是 skill —— 显式调用做蒸馏

第三方 skill

gstack /review、/ship 装进你的 Engine, 受你的 Gate 约束

第三方 skill = 装进你 Engine 的 app —— 它们在你的轨道上跑, 标准冲突时你的 Gate 拦住它。

MCP = 外部能力接入 (Agent 的工具箱扩展)



原则：能用 CLI 解决的不用 MCP
MCP 只接 CLI 做不到的领域逻辑

解决什么

访问 CLI 之外的系统：数据库 / CI / 云资源 / 专业领域工具

首要安全机制

给工具 = 给权限。permissions.deny 是第一道防线

本课定位

支撑角色，不展开配置。Engine SDLC 实弹中按需使用

一句话：能用 CLI 解决的不用 MCP —— Claude Code 自带 bash 与文件操作，大部分事 CLI 就够了。

Context = 上下文工程 (Agent 的工作记忆)



Context Engineering 三原则

1 · 最重要的放最前

对抗 attention decay —— Principles 居首

2 · 不需要的不加载

渐进式 / lazy load, 节省预算

3 · 定期清理过期

staleness detection, 丢掉陈旧内容

塞太多 → 质量下降 (bloated CLAUDE.md) 。这也是蒸馏的核心动力之一。

Sessions = 会话生命周期 (Agent 的作息规律)



persistent state (跨 session 持久化) : DDD · Memory · STATE.md

跨 session = L4 基础

记住上次的能力来自 Harness，不是模型自带

Engine/State 持久化

跑到 BUILD 下班 → 明天新 session 续跑

Knowledge/Memory 积累

经验跨 session 沉淀，越用越懂你的项目

每次新 session，agent 都是白纸 —— SessionStart 的注入质量 = 这个 session 的智力上限。

总结 · 六大能力 × 两个模块

模块如何组合使用基座能力 —— 全景对照

能力	Knowledge 怎么用	Engine 怎么用
Rules	Governance 产出 (principles / rules)	—
Hooks	Capture (SessionEnd) + Retrieve (SessionStart)	Gate (PreCommit) + Stage 触发
Skills	distill / evolve	Stage 执行逻辑
MCP	— (未来可接向量库)	领域工具接入
Context	注入策略 (什么放前、什么 lazy load)	State 占位管理
Sessions	跨 session 记忆积累	跨 session 状态恢复

零件是通用的，设计是你的。你不学「怎么写 hook」，你学「模块为什么需要它、放哪、触发什么」。

休息 15 分钟 ▶ 回来开始建 Knowledge 模块 (hooks + skills + context + sessions 组合成有生命力的系统)